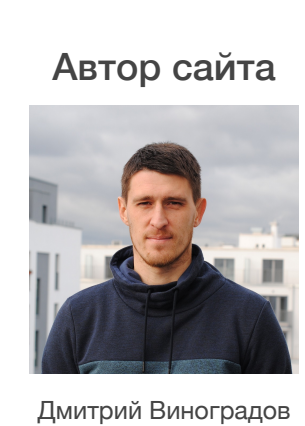
Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Курс
Jetpack Compose

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 086 подписчиков

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 12. ViewModel

14 декабря 2022

В этом уроке используем ViewModel для хранения данных и используем StateFlow вместо State.

В прошлых уроках мы для простоты примеров хранили State в Activity. Но в реальном приложении для хранения данных экрана обычно используется ViewModel. Давайте посмотрим, как это работает в Compose.

Чтобы Composable функции смогли получать ViewModel, необходимо добавить зависимость:

```
implementation 'androidx.lifecycle:lifecycle-viewmodel-compose:2.5.1'
```

В качестве примера снова возьмем счетчик кликов на экране HomeScreen:

```
@Composable
fun HomeScreen() {
   Text(
       text = "Clicks: 0",
       modifier = Modifier.clickable(onClick = { })
   )
}
```

Пока что он показывает 0 и не реагирует на нажатия. Нам нужен State, в котором будет хранится значение счетчика. Этот State будет хранится в ViewModel.

Создаем модель для экрана HomeScreen:

```
class HomeViewModel: ViewModel() {

   val counter = mutableStateOf(0)

}
```

В ней создаем State для счетчика.

Добавляем модель, как параметр в функцию HomeScreen:

```
import androidx.lifecycle.viewmodel.compose.viewModel

@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = viewModel()
) {
   // ...
}
```

В качестве значения по умолчанию вызываем функцию viewModel(). Она сделает всю работу по созданию и получению модели. И при вызове HomeScreen из Activity нам ничего не надо будет передавать:

```
// …
setContent {
   HomeScreen()
}
// …
```

Таким образом у нас в HomeScreen появился доступ к модели и мы можем из нее взять State для счетчика:

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue

@Composable
fun HomeScreen(
    homeViewModel: HomeViewModel = viewModel()
) {
    var counter by homeViewModel.counter
    Text(
        text = "Clicks: $counter",
        modifier = Modifier.clickable(onClick = { counter++ })
    )
}
```

Используем by чтобы работать с значением State как с обычной переменной. Отображаем значение в Text и увеличиваем на 1 по клику.

В целом вариант получился рабочий, но не очень хороший. У нас в модели поле counter имеет тип MutableState. Следовательно, мы можем напрямую менять это значение из UI. А это, как мы уже ранее обсуждали - не очень хорошо. UI должен не сам менять State, а вызывать метод, который будет это делать. Это снижает вероятность непредсказуемого поведения, повышает тестируемость кода и дает возможность добавить логику в процесс изменения значения State.

Перепишем модель:

```
class HomeViewModel: ViewModel() {

   private val _counter = mutableStateOf(0)
   val counter: State<Int> = _counter

   fun onCounterClick() {
       _counter.value++
   }
}
```

Теперь это больше похоже на то, что мы делали ранее. Модель наружу отдает не MutableState, а State, который не позволяет напрямую менять значение. Для изменения значения теперь есть метод onCounterClick.

Перепишем HomeScreen:

```
import androidx.compose.runtime.getValue

@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = viewModel()
) {
   val counter by homeViewModel.counter
   Text(
       text = "Clicks: $counter",
       modifier = Modifier.clickable(onClick = homeViewModel::onCounterClick)
   )
}
```

Читаем значение из State и отображаем его на экране. А по клику вызываем метод модели onCounterClick. Этот метод увеличит значение в State, что приведет к перезапуску HomeScreen и отображению нового значения.

Все работает точно так же, как мы делали раньше, только State теперь находится в модели, а не в Activity.

Flow, Rx, LiveData

В модели данные можно хранить не только в State, но и в более привычном для нас формате: Flow, Rx или LiveData. Для этих типов данные есть специальные конвертеры в State.

Рассмотрим на примере Flow:

```
class HomeViewModel: ViewModel() {

   private val _counter = MutableStateFlow(0)
   val counter: StateFlow<Int> = _counter

   fun onCounterClick() {
       _counter.value++
   }
}
```

В модели вместо компоузовского State используем корутиновский StateFlow

В HomeScreen при этом будет минимум изменений:

```
@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = viewModel()
) {
   val counter by homeViewModel.counter.collectAsState()
   Text(
       text = "Clicks: $counter",
       modifier = Modifier.clickable(onClick = homeViewModel::onCounterClick)
   )
}
```

Функция collectAsState подписывается на StateFlow и предоставляет State. Каждый раз, когда значение в StateFlow меняется, обновляется и значение State.

Получается такая схема подписки:
Composable -> State -> StateFlow

Composable читает значение из State, и тем самым подписывается на него, чтобы перезапуститься при изменении значения. А State подписывается на StateFlow. В итоге изменение значения в StateFlow будет приводить к перезапуску Composable функции.

ViewModel в иерархии Composable функций

Сейчас наш пример слишком простой. В реальном приложении мы вряд ли будем размещать весь код в HomeScreen. Вместо этого мы компоненты экрана выносим в отдельные Composable функции и их уже вызываем в HomeScreen.

Давайте вынесем код счетчика в отдельную функцию, и сделаем это не очень правильно:

```
@Composable
fun HomeScreen() {
   ClickCounter()
}

@Composable
fun ClickCounter(
   homeViewModel: HomeViewModel = viewModel()
) {
   val counter by homeViewModel.counter.collectAsState()
   Text(
       text = "Clicks: $counter",
       modifier = Modifier.clickable(onClick = homeViewModel::onCounterClick)
   )
}
```

Так как нам в ClickCounter нужен State и метод из модели, то мы просто получаем и используем модель прямо в этой функции. Мы можем так сделать в любой функции экрана. Везде мы получим один и тот же экземпляр модели. Выглядит удобно. Но так делать не рекомендуется.

Представим, что у нас на других экранах тоже есть счетчики кликов. Мы могли бы функцию ClickCounter переместить в какой-то базовый package проекта и оттуда использовать на разных экранах. Но она жестко привязана к HomeViewModel. Другие экраны со своими моделями не смогут контролировать State счетчика.

Поэтому работать с моделью рекомендуется только в самой верхней Composable функции экрана, а вниз по иерархии уже передавать значения и методы модели.

В официальной документации есть отдельный пункт про это:

Warning: Don't pass ViewModel instances down to other composable functions. Doing so couples the composable function with the ViewModel type, making it less reusable and harder to test and preview. Also, there would be no clear single source of truth (SSOT) that manages the ViewModel instance. Passing the ViewModel down allows multiple composables to call ViewModel functions and modify its state, making bugs harder to debug. Instead, follow UDF best practices and pass down just the necessary state. Likewise, pass the propagating events up until they reach the ViewModel's composable SSOT. That is the SSOT which handles the event and calls the corresponding ViewModel methods.

Давайте перепишем код, чтобы избавить ClickCounter от привязки к конкретной модели:

```
@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = viewModel()
) {
   val counter by homeViewModel.counter.collectAsState()
   ClickCounter(
       count = counter,
       onCounterClick = homeViewModel::onCounterClick
   )
}

@Composable
fun ClickCounter(
   count: Int,
   onCounterClick: () -> Unit
) {
   Text(
       text = "Clicks: $count",
       modifier = Modifier.clickable(onClick = onCounterClick)
   )
}
```

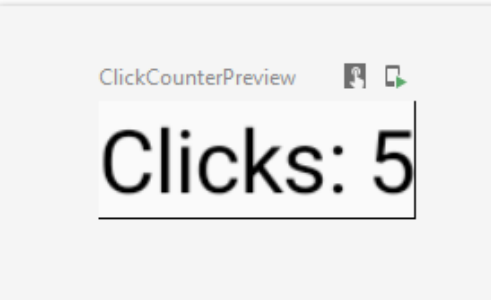
HomeScreen берет все необходимые данные из модели и передает в ClickCounter. Теперь ClickCounter не привязан к HomeViewModel и может быть использован на любых других экранах.

А также для такой функции ClickCounter легко сделать Preview:

```
@Preview(showBackground = true)
@Composable
fun ClickCounterPreview() {
   ClickCounter(
       count = 5,
       onCounterClick = {}
   )
}
```

Передаем число и пустую лямбду.

Результат:



Clicks: 5

А если бы функция ClickCounter на вход ожидала не просто данные и лямбды, а модель, то пришлось бы костылить, чтобы увидеть в Preview желаемые значения.

UiState

Если экран достаточно сложный, то в модели получится много различных StateFlow. Их всех можно объединить в один StateFlow и работать с ним. Чтобы посмотреть, как это работает, давайте добавим на экран еще один компонент с отдельным State, а потом попробуем объединить два State в один.

На экране HomeScreen к счетчику кликов добавим простой чекбокс, который будет включать какую-то фичу. Создаем функцию для этого чекбокса:

```
@Composable
fun EnableFeature(
   enabled: Boolean,
   onEnabledChange: (Boolean) -> Unit
) {
   Row(verticalAlignment = CenterVertically) {
       Checkbox(checked = enabled, onCheckedChange = onEnabledChange)
       Text("enable feature")
   }
}
```

На вход принимает значение чекбокса и лямбду для кликов.

Дополним модель:

```
class HomeViewModel: ViewModel() {

   private val _counter = MutableStateFlow(0)
   val counter: StateFlow<Int> = _counter

   fun onCounterClick() {
       _counter.value++
   }


   private val _enabled = MutableStateFlow(false)
   val enabled: StateFlow<Boolean> = _enabled

   fun setEnabled(enabled: Boolean) {
       _enabled.value = enabled
   }

}
```

Создаем новый StateFlow для хранения значения чекбокса и метод для изменения этого значения.

Собираем все это в HomeScreen:

```
@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = viewModel()
) {
   val counter by homeViewModel.counter.collectAsState()
   val enabled by homeViewModel.enabled.collectAsState()

   Column {
       ClickCounter(
           count = counter,
           onCounterClick = homeViewModel::onCounterClick
       )
       EnableFeature(
           enabled = enabled,
           onEnabledChange = homeViewModel::setEnabled
       )
   }
}
```

Из модели мы получаем два State, и передаем их значения далее в функции ClickCounter и EnableFeature.

Когда таких State всего два, это смотрится нормально. Но если их 20, то будет уже не очень красиво. Попробуем объединить два этих State в один. Для этого создадим отдельный класс в котором будут храниться значения:

```
data class HomeScreenUiState(
   val count: Int = 0,
   val enabled: Boolean = false
)
```

count для значения счетчика и enabled - для чекбокса

В модели заменяем два State на один:

```
class HomeViewModel : ViewModel() {

   private val _uiState = MutableStateFlow(HomeScreenUiState())
   val uiState: StateFlow<HomeScreenUiState> = _uiState

   fun onCounterClick() {
       _uiState.update {
           it.copy(count = it.count + 1)
       }
   }

   fun setEnabled(enabled: Boolean) {
       _uiState.update {
           it.copy(enabled = enabled)
       }
   }
}
```

Вместо StateFlow<Int> и StateFlow<Boolean> у нас теперь StateFlow<HomeScreenUiState>. Оба метода меняют соответствующие поля в этом State.

В HomeScreen заменяем два State на один:

```
@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = viewModel()
) {
   val uiState by homeViewModel.uiState.collectAsState()

   Column {
       ClickCounter(
           count = uiState.count,
           onCounterClick = homeViewModel::onCounterClick
       )
       EnableFeature(
           enabled = uiState.enabled,
           onEnabledChange = homeViewModel::setEnabled
       )
   }
}
```

Читаем значение HomeScreenUiState из State и раскидываем значения по функциям.

Preview для HomeScreen

Функция HomeScreen принимает на вход модель

```
@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = viewModel()
) {
   // ...
}
```

Как мы уже обсуждали чуть ранее, это значит, что мы не сможем сделать для нее Preview простым способом. Но что делать, если хочется видеть превью всего экрана, а не только отдельных его компонентов?

Чтобы решить эту проблему, можно сделать небольшой трюк. Мы создаем еще одну версию функции HomeScreen:

```
@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = viewModel()
) {
   val uiState by homeViewModel.uiState.collectAsState()
   HomeScreen(
       uiState = uiState,
       onCounterClick = homeViewModel::onCounterClick,
       onEnabledChange = homeViewModel::setEnabled
   )
}

@Composable
fun HomeScreen(
   uiState: HomeScreenUiState,
   onCounterClick: () -> Unit,
   onEnabledChange: (Boolean) -> Unit
) {
   Column {
       ClickCounter(
           count = uiState.count,
           onCounterClick = onCounterClick
       )
       EnableFeature(
           enabled = uiState.enabled,
           onEnabledChange = onEnabledChange
       )
   }
}
```

Мы можем иметь две функции с одинаковым именем HomeScreen, потому что их параметры различаются.

Первая функция HomeScreen, как и раньше, на вход принимает модель. А вот дальше она достает из модели все необходимое и передает второй функции HomeScreen.

Вторая функция HomeScreen принимает на вход данные и раскидывает их по компонентам.

И для этой второй функции мы и можем сделать Preview, чтобы увидеть, как будет выглядеть весь экран со всеми компонентами

```
@Preview(showSystemUi = true)
@Composable
fun HomeScreenPreview() {
   HomeScreen(
       uiState = HomeScreenUiState(count = 5, enabled = true),
       onCounterClick = {},
       onEnabledChange = {}
   )
}
```

В Preview мы вызываем вторую функцию HomeScreen и передаем ей данные и пустые лямбды.

Dagger

В примере этого урока мы работали с простой моделью с пустым конструктором, которая не использует никакие другие объекты. Но это крайне редкий случай в реальном приложении. Обычно у нас в модель приходят всевозможные репозитории.

Рассмотрим пример такой модели в приложении с даггером:

```
class HomeViewModel(
   private val repository: SomeRepository
) : ViewModel() {
   // ...
}
```

Модель ожидает получить на вход репозиторий

Попробуем получить эту модель в Composable функции:

```
@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = viewModel()
) {
   // ...
}
```

Вместо модели мы получим крэш: Cannot create an instance of class ru.startandroid.mycomposeapplication.HomeViewModel

Это стандартное поведение системы. Она не может создать модель, у которой не пустой конструктор. От нас требуется фабрика, которую система будет использовать для создания модели.

Мы сейчас не будем разбираться, как создавать такую фабрику. Об этом есть статьи в Интернет или отдельный урок в курсе Dagger.

В Activity получаем фабрику от компонента и используем ее при вызове Composable функции

```
class MainActivity : ComponentActivity() {

   override fun onCreate(savedInstanceState: Bundle?) {
       super.onCreate(savedInstanceState)

       val mainComponent = (application as App).appComponent
           .getMainComponent()
           .create(this)

       val viewModelFactory = mainComponent.getViewModelFactory()

       setContent {
           HomeScreen(
               homeViewModel= viewModel(factory = viewModelFactory)
           )
       }
   }
}
```

Мы используем фабрику в функции viewModel() чтобы система смогла создать модель, которая пойдет на вход Composable функции.

Код получения фабрики у вас будет свой, в зависимости от того, как у вас настроен даггер.

Hilt

Если вы используете Hilt, то все гораздо проще.

Есть модель:

```
@HiltViewModel
class HomeViewModel @Inject constructor(
   private val repository: SomeRepository
) : ViewModel() {
   // ...
}
```

Есть функция:

```
@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = viewModel()
) {
   // ...
}
```

Есть Activity:

```
@AndroidEntryPoint
class MainActivity : ComponentActivity() {
   override fun onCreate(savedInstanceState: Bundle?) {
       super.onCreate(savedInstanceState)
       setContent {
           HomeScreen()
       }
   }
}
```

В отличие от даггера нам не нужно создавать никаких фабрик. Hilt все сделает за нас. Главное, чтобы он знал, как создать SomeRepository. Вызов viewModel() в Composable функции без проблем создаст модель.

collectAsStateWithLifecycle

Если в вашей модели есть продолжительный Flow (например - получение текущих координат), на который вы подписываетесь в UI, то имеет смысл вместо collectAsState использовать функцию collectAsStateWithLifecycle. Она будет держать подписку на Flow с соблюдением Lifecycle.

Чтобы ее использовать, добавьте зависимость

```
implementation "androidx.lifecycle:lifecycle-runtime-compose:2.6.0-alpha03"
```

Полезные ссылки

В официальной документации есть пара разделов, в которых можно подробно почитать про то, о чем мы говорили в этом уроке.

https://developer.android.com/jetpack/compose/state-hoisting#screen-ui

https://developer.android.com/topic/architecture/ui-layer/stateholders#dependencies

---

Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

Назад Вперёд

Комментарии

# **RE: Урок 12. ViewModel** — **Максим** 15.01.2023 01:36
Спасибо, пример с даггером меня успокоил - с самого начала был вопрос про assisted inject.

Кстати, даггер вроде тоже позволяет прокидывать зависимости через inject constructor

Ответить

# **Сергей** — **Sergey** 26.01.2023 16:21
Не совсем понятно про создание ViewModel в данном случае. На сколько я понял из приведенного Вами примера - ViewModel будет создаваться при перевороте экрана каждый раз заново? Если нет, то почему?

Ответить

# **RE: Сергей** — **Dmitry Vinogradov** 03.02.2023 18:56
Уточните, про какой именно пример речь?
ViewModel создана, чтобы пережить поворот экрана. И Compose не меняет это поведение модели.

Ответить

# **RE: Урок 12. ViewModel** — **alex** 09.02.2023 15:06
Добрый день. Подскажите пожалуйста как передать новое значение через callback в state?

```
class MainViewModel: ViewModel(){

private val _text = MutableStateFlow("")
val text: StateFlow = _text

fun onSetText(newText : String){
_text.value = newText
}
}


@Composable
fun MainScreen(mainViewModel: MainViewModel = viewModel()){

val text by mainViewModel.text.collectAsState()

Column(modifier = Modifier.fillMaxSize()) {
elemB(text, mainViewModel::onSetText)
}

}

@Composable
fun elemB(text: String, callback: (text:String) -> Unit){

Log.d("MyLog", "elemB")
Text("elemB $text")

Button(onClick = callback ) {
Text("elemB")
}

}

callback("Что-то")
```

Ответить

# **RE: Урок 12. ViewModel** — **Dmitry Vinogradov** 20.02.2023 21:55
Если передаете по нажатию на кнопку, то так:
Button(onClick = { callback("new value") })

Ответить

# **Тяжелый Flow** — **Татьяна** 07.06.2023 18:08
"Если в вашей модели есть тяжелый Flow" А можете привести пример, что имеете ввиду под "тяжелым Flow"?

Ответить

# **RE: Тяжелый Flow** — **Dmitry Vinogradov** 09.06.2023 16:46
Некорректно сформулировал. Переделал на "продолжительный Flow (например - получение текущих координат)"

Ответить

# **RE: Урок 12. ViewModel** — **Николай** 13.02.2024 09:24
всё течёт всё меняется
implementation 'androidx.lifecycle:lifecycle-viewmodel-compose:2.5.1'
V
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.7.0")

Ответить

# **Recomposition** — **Vitaly** 16.02.2024 18:11
Подскажите, почему, например, в блоке про uiState, при клике на счетчике, например, вызывается функция EnableFeature, хотя чекбокс не менялся, лямбда одна и та же, входные параметры не менялись, модель одна и та же, как в этом случае работает принцип рекомпозиции?

Ответить

# **Recomposition** — **Vitaly** 17.02.2024 13:37
нашел проблему в collectAsState и переходе на Flow, может стоит по возможности избегать этой реализации, чтобы улучшить работу рекомпозиции, хотел бы узнать мнение по лучшей практике

Ответить

# **Recomposition** — **Vitaly** 16.02.2024 18:24
Заметил, что если в функции передать State, тогда такой проблемы нет и только одна из них вызывается

Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.